

Placing and Routing Qubits on a 20-Qubit Heavy-Hex Chip

QSITE 2026 · Computational Track · Team telequantum · Authored by Aziz Chebil

283.5

baseline score

68.5

our score (~76%)

3 / 6

benchmarks provably optimal

6 / 6

valid under the official scorer

Problem

Map each program's logical qubits onto a 20-qubit heavy-hex graph (1–3 neighbours per qubit) and insert SWAPs so every two-qubit gate acts on an edge. Program order is fixed (stripping SWAPs must recover the exact program). **Score = SWAPs + 0.5 × depth**, where depth comes from ASAP layer packing; lower is better.

Approach

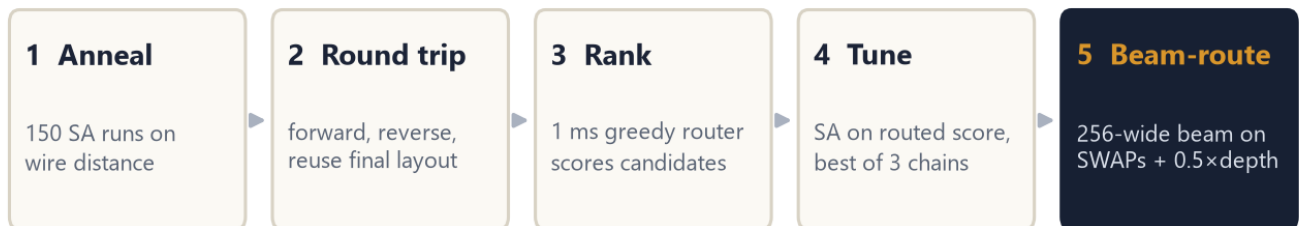


Figure 1. Pipeline. Steps 1–3 search placements broadly and cheaply; steps 4–5 spend compute on the true objective.

Step	What it does	Why
1. SA placement	150 annealing runs minimise Σ hardware distance over all 2Q pairs; moves swap two qubits or jump to a free physical qubit	puts partners close; free-qubit moves explore the whole chip
2. Round trip	route forward, route the reversed program from the final layout, restart from where qubits ended (SABRE-style, 4 rounds)	layouts adapt to where the program actually needs qubits
3. Greedy rank	look-ahead router: SWAP minimising current + decayed distance of the next 20 gates	~1 ms per route, scores every candidate cheaply
4. Tune on real score	SA whose cost is the routed score (width-8 beam, cached); 3 independent chains, best kept	distance is only a proxy; one chain often stalls
5. Beam routing	keeps up to 256 partial routings, tracks each qubit's last layer (exactly as the scorer schedules), ranks by swaps + 0.5×depth + look-ahead	chooses which qubit moves and along which path, so SWAPs run in parallel

Results

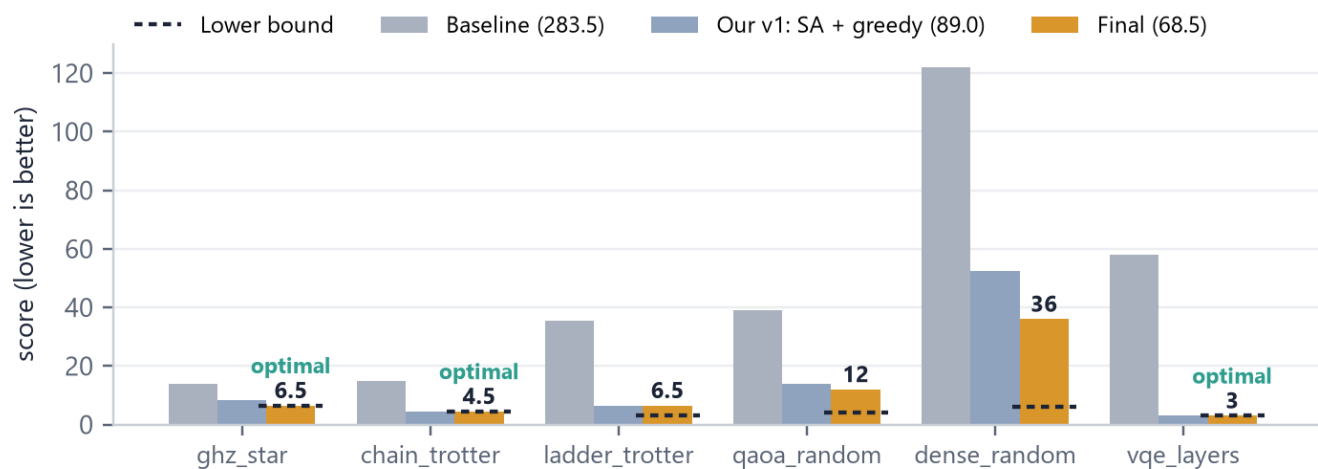


Figure 2. Score per benchmark. Dashed ticks are lower bounds: $0.5 \times$ the program's own dependency depth (zero SWAPs), and for ghz_star a hand proof (below). Deterministic (fixed seed); reproduce with `python evaluate.py`.

Why a depth-aware router matters

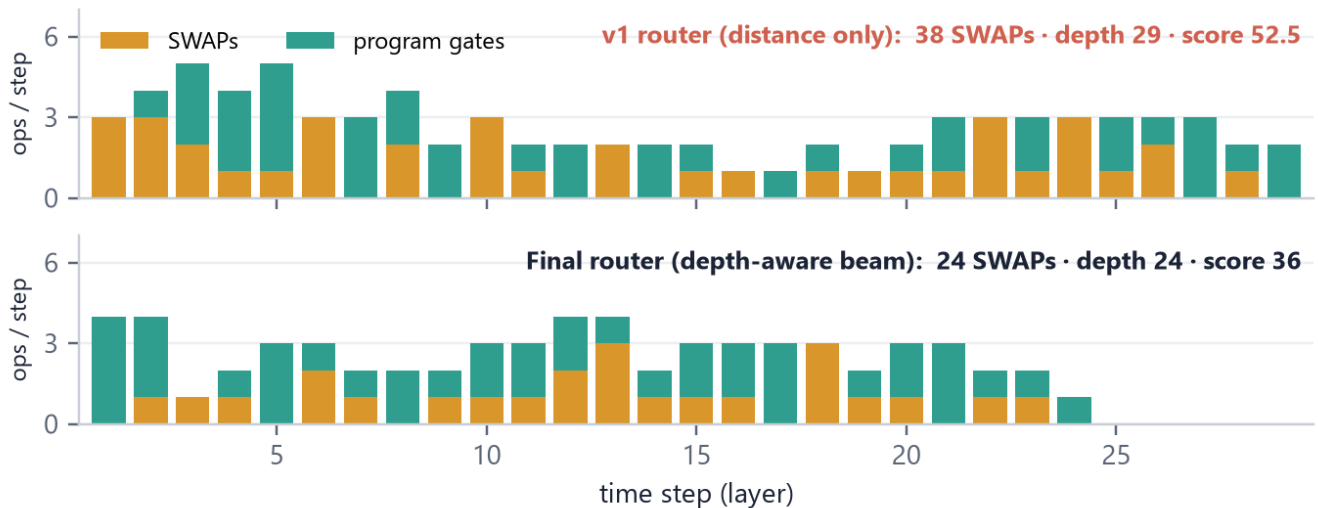


Figure 3. Operations per time step on `dense_random` (14 qubits, 40 gates), real routings. The v1 router picked the SWAP that most reduced distance and ignored when it could run, so SWAPs queued on busy qubits. The beam router scores depth exactly and packs more work into each step: 14 fewer SWAPs and 5 fewer steps.

What each idea bought

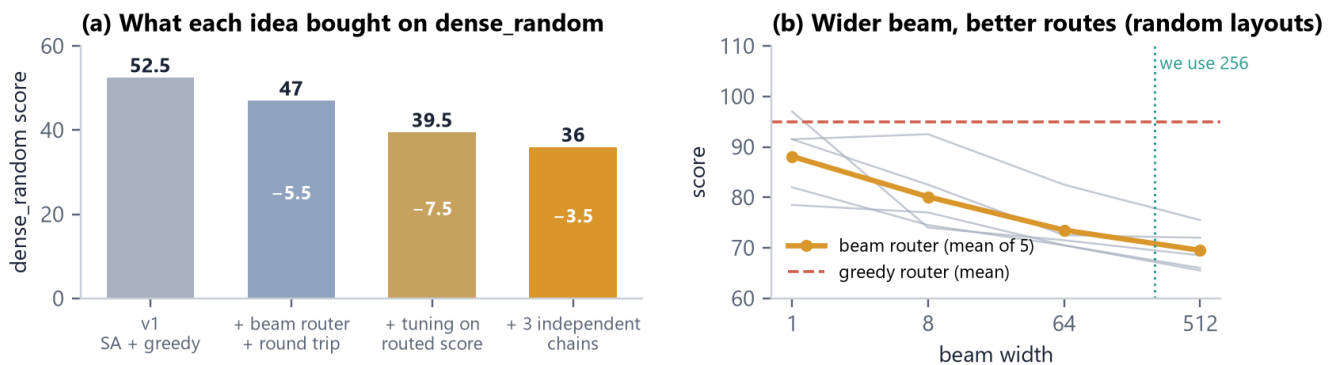


Figure 4. (a) Cumulative gains on `dense_random`, measured as each component was added. The largest single gain came from tuning placement on the routed score rather than on the distance proxy. (b) Beam width vs score on 5 random layouts of `dense_random` (grey: each layout; amber: mean). Returns diminish past a few hundred; 256 costs ~ 150 ms per route, so we use it only for the final pass and width 8 inside the tuning loop.

How close to optimal?

- **chain_trotter, vqe_layers**: zero SWAPs at the program's own depth, so no solution can score lower.
- **ghz_star**: the hub must interact with 7 qubits but has at most 3 neighbours, so it must move at least twice; its gates and moves are serial, so $\text{depth} \geq 7 + 2 = 9$ and $\text{score} \geq 2 + 4.5 = 6.5$, which we achieve.
- **ladder, qaoa, dense**: the gap to the bound is structural (the chip has no 4-cycles; random circuits touch every qubit), but the bound is loose, so some headroom likely remains.

What didn't work, and robustness

- **Sideways SWAPs** (moves that don't shorten the current gate but help the next 3) won 10 / lost 9 on random layouts; inside the tuning loop they made results worse and $\sim 2.5\times$ slower. Kept only as an extra final-pass option.
- **Seed choice**: another seed scores 68.0; we kept a fixed seed rather than tune to the test set.
- **Unseen programs**: 1-qubit, single-gate, mixed 1Q/2Q and random 16–20-qubit programs all validate; runtime 1–23 s per benchmark with a 60 s safety cap. Stretch goals were not attempted (no scored baseline was provided).